

Linear Decoding Reveals Low-Dimensional Time-Series Structure in Language Model Representations

Prateek Humane

Abstract

We investigate whether pretrained language models encode latent structure sufficient to generate realistic time series without any paired supervision. Using hidden states from a pretrained Qwen3-0.6B model processing WikiText, we train a simple per-timestep linear map $\hat{\mathbf{y}}_{i,t} = \mathbf{W}\mathbf{h}_{i,t} + b$ to produce time series outputs matched to real sequences from GiftEval via an EM-style nearest-neighbor objective augmented with a shift-invariant spectral diversity penalty.

We find that: (1) pretrained representations enable significantly better alignment with real time series than random or untrained baselines, with $2\text{--}3\times$ lower nearest-neighbor distance; (2) diversity requires *both* trained weights and meaningful text input—neither component alone is sufficient; (3) pretrained representations are highly anisotropic, with over 80% of variance concentrated in a single principal direction, yielding an effective dimensionality of $\sim 3\text{--}6$ in a 28,672-dimensional space; and (4) decoded outputs are spectrally aligned with real time series, concentrating 90% of energy in the lowest 10% of frequencies. These results suggest that language pretraining induces a low-dimensional dynamical manifold in representation space that can be linearly projected into realistic temporal signals.

1 Introduction

Recent work has demonstrated that language models pretrained on text can be successfully fine-tuned for time series forecasting, often outperforming domain-specific architectures. This raises a fundamental question: *why does language pretraining help with time series?* One hypothesis is that autoregressive language modeling induces internal representations that capture general sequential dynamics—temporal smoothness, periodicity, trend—that transfer across modalities.

We approach this question empirically with a minimal experimental setup:

- **No paired data:** we never show the model text–time-series pairs.
- **Linear map:** a single weight vector $\mathbf{W} \in \mathbb{R}^{1 \times D}$ applied per timestep.
- **Unpaired matching:** predictions are trained to match *some* real time series via nearest-neighbor alignment.

Despite this extreme simplicity, we find that a pretrained language model processing natural text produces hidden states from which a linear map can decode outputs that closely resemble real-world time series data. Through a systematic 2×2 ablation (architecture $\times$ input), representation geometry analysis, and spectral characterization, we establish that this capacity requires the specific combination of learned weights and meaningful text input.

2 Setup

2.1 Models

All experiments use the Qwen3-0.6B architecture: 28 transformer layers with hidden dimension $d = 1024$ ($\sim 0.6\text{B}$ parameters). We compare two model variants:

- **PT** (Pretrained): Qwen/Qwen3-0.6B, pretrained on text corpora. This model has never been exposed to numerical time series data.
- **RandomInit**: The identical Qwen3-0.6B architecture with freshly initialized random weights (PyTorch default initialization). Never trained on any data.

2.2 Data

Text. WikiText-103 train split, tokenized with the Qwen3-0.6B tokenizer and chunked into sequences of exactly $T = 512$ tokens. Training set: sequences 0–1919 ($N_{\text{train}} = 1920$). Held-out set: sequences 2000–4999 ($N_{\text{held}} = 3000$), never seen during mapper training.

Time series. GiftEval validation set (Salesforce/GiftEval on HuggingFace). Non-overlapping windows of $T = 512$ timesteps, z -score normalized per window:

$$y_t \leftarrow \frac{y_t - \mu}{\max(\sigma, 10^{-6})}, \quad \mu = \frac{1}{T} \sum_{t=1}^T y_t, \quad \sigma = \sqrt{\frac{1}{T} \sum_{t=1}^T (y_t - \mu)^2} \quad (1)$$

Windows with $\sigma < 10^{-6}$ (constant values) or containing NaN/Inf are excluded. Evaluation bank: $M = 10,000$ clean windows $\{\mathbf{Y}_j\}_{j=1}^M$.

Random tokens. For ablation experiments, we generate sequences of $T = 512$ token IDs sampled uniformly from the full vocabulary ($|\mathcal{V}| = 151,936$).

2.3 Hidden State Extraction

For each input sequence $\mathbf{x}_i = (x_{i,1}, \dots, x_{i,T})$, we perform a forward pass through the model and capture the hidden state output at every transformer layer $\ell \in \{0, 1, \dots, 27\}$ using forward hooks:

$$\mathbf{h}_i^{(\ell)} = \text{Layer}_\ell(\mathbf{h}_i^{(\ell-1)}) \in \mathbb{R}^{T \times d} \quad (2)$$

where $\mathbf{h}_i^{(-1)}$ is the embedding output.

For the concatenated-layer representation, we concatenate all 28 layers along the hidden dimension:

$$\mathbf{h}_{i,t} = [\mathbf{h}_{i,t}^{(0)}; \mathbf{h}_{i,t}^{(1)}; \dots; \mathbf{h}_{i,t}^{(27)}] \in \mathbb{R}^D, \quad D = 28 \times 1024 = 28,672 \quad (3)$$

This produces a representation $\mathbf{H}_i \in \mathbb{R}^{T \times D}$ for each input sequence i .

3 Method: Linear Decoding with EM Matching

3.1 Linear Map

We apply a single linear layer independently at each timestep:

$$\hat{y}_{i,t} = \mathbf{W}\mathbf{h}_{i,t} + b \quad (4)$$

where $\mathbf{W} \in \mathbb{R}^{1 \times D}$ and $b \in \mathbb{R}$. For the concatenated representation with $D = 28,672$, the total number of learnable parameters is $D + 1 = 28,673$.

3.2 Output Normalization

Predicted sequences are z -score normalized before evaluation and loss computation:

$$\hat{\mathbf{y}}_i \leftarrow \frac{\hat{\mathbf{y}}_i - \text{mean}(\hat{\mathbf{y}}_i)}{\max(\text{std}(\hat{\mathbf{y}}_i), 10^{-4})} \quad (5)$$

This ensures predictions have approximately zero mean and unit variance, matching the normalized time series targets.

3.3 Training Objective

Since there is no paired text–time-series supervision, we cannot directly compute a per-sample loss. Instead, we use an EM-style hard-matching objective that iterates between assigning each prediction to its nearest real time series and minimizing the distance to that assignment.

Algorithm 1 EM-Style Training with Diversity Penalty

```

1: for each epoch do
2:   for each mini-batch of hidden states  $\{\mathbf{H}_i\}_{i=1}^B$  do
3:     Forward: Compute predictions  $\hat{\mathbf{y}}_i = \mathbf{W}\mathbf{H}_i + b$ , normalize each  $\hat{\mathbf{y}}_i$ 
4:     E-step: Sample  $K = 128$  real time series  $\{\mathbf{Y}_k\}_{k=1}^K$  from the bank
5:     for each prediction  $\hat{\mathbf{y}}_i$  do
6:        $j_i^* = \operatorname{argmin}_{k \in \{1, \dots, K\}} \frac{1}{T} \|\hat{\mathbf{y}}_i - \mathbf{Y}_k\|^2$  ▷ Nearest neighbor in sample
7:     end for
8:     M-step: Compute MSE loss to matched targets:

```

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{T} \|\hat{\mathbf{y}}_i - \mathbf{Y}_{j_i^*}\|^2$$

```

9:     Diversity: Compute PSD diversity penalty (Eq. 7)
10:    Update:  $\theta \leftarrow \theta - \eta \nabla_{\theta} (\mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{L}_{\text{div}})$ 
11:  end for
12: end for

```

Note that the E-step targets are computed with `torch.no_grad()` (stop-gradient), so gradients flow only through the M-step MSE. At each iteration, a fresh random sample of $K = 128$ time series is drawn from the full bank of $M = 10,000$, providing stochastic coverage of the target distribution.

3.4 PSD Diversity Penalty

Without regularization, the linear map collapses to producing a single dominant output shape (mode collapse), since the EM objective rewards matching *any* real time series and the easiest solution is to match the most common pattern.

We introduce a *shift-invariant* diversity penalty based on the power spectral density (PSD). For a predicted sequence $\hat{\mathbf{y}}_i \in \mathbb{R}^T$, its PSD is the squared magnitude of the discrete Fourier transform:

$$\text{PSD}(\hat{\mathbf{y}}_i) = |\text{FFT}(\hat{\mathbf{y}}_i)|^2 \in \mathbb{R}^{T/2+1} \quad (6)$$

The diversity penalty penalizes cosine similarity between PSD vectors within a batch:

$$\mathcal{L}_{\text{div}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \cos_sim(\text{PSD}(\hat{\mathbf{y}}_i), \text{PSD}(\hat{\mathbf{y}}_j)) \quad (7)$$

where $\mathcal{P} = \{(i, j) : i \neq j\}$ is the set of all pairs in the batch.

This penalty is shift-invariant: two temporal translations of the same signal have identical PSDs and will be maximally penalized. The total training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{L}_{\text{div}} \quad (8)$$

We sweep $\lambda \in \{0, 0.5, 0.7, 1.0, 1.5\}$ and find $\lambda = 0.5$ optimal, balancing match quality against diversity.

Parameter	Value
Optimizer	Adam ($\eta = 10^{-3}$)
Batch size (text)	$B = 32$ sequences
TS comparison pool	$K = 128$ per batch (sampled from $M = 10,000$)
Epochs	100
Diversity weight	$\lambda = 0.5$
Sequence length	$T = 512$
Hidden dimension	$D = 28 \times 1024 = 28,672$ (concatenated)
Total parameters	28,673
Random seed	1 (selected from stability analysis over seeds 0–4)

Table 1: Training hyperparameters.

3.5 Training Details

4 Evaluation Metrics

Nearest Neighbor Distance (NN dist). For each prediction $\hat{\mathbf{y}}_i$, we compute the MSE to every time series in the full evaluation bank and take the minimum:

$$d_i = \min_{j \in \{1, \dots, M\}} \frac{1}{T} \|\hat{\mathbf{y}}_i - \mathbf{Y}_j\|^2 \quad (9)$$

We report both the *raw* mean $\bar{d} = \frac{1}{N} \sum_i d_i$ and the *deduplicated* mean (averaging only over unique matches).

Unique Matches. The number of distinct real time series that are the nearest neighbor of at least one prediction:

$$\text{Unique} = \left| \left\{ j_i^* : j_i^* = \underset{j}{\operatorname{argmin}} \|\hat{\mathbf{y}}_i - \mathbf{Y}_j\|^2, i = 1, \dots, N \right\} \right| \quad (10)$$

Cluster Coverage. Real time series are clustered into $C = 20$ groups via K -means on their autocorrelation function (ACF) profiles at lags $0, 1, \dots, 31$:

$$\text{ACF}(\mathbf{y}, \tau) = \frac{\sum_{t=1}^{T-\tau} (y_t - \bar{y})(y_{t+\tau} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2} \quad (11)$$

Coverage is the number of clusters containing at least one matched time series.

Match Entropy. The normalized entropy of the match distribution:

$$H = -\frac{\sum_j p_j \log p_j}{\log N}, \quad p_j = \frac{|\{i : j_i^* = j\}|}{N} \quad (12)$$

where $H = 1$ indicates uniform assignment across all predictions and $H \rightarrow 0$ indicates mode collapse.

Top- K Fair Comparison. To compare models with different numbers of unique matches, we evaluate at matched K : for each model, take the best- K unique matches (deduplicated) and compare mean NN distance. This ensures we compare quality at the same diversity level.

5 Results

5.1 Pretrained Models Encode Decodable Temporal Structure

We first establish the basic finding using single-layer extraction (layer 8, $D = 1024$) without a diversity penalty.

Method	NN Dist	Description
PT (pretrained)	0.716	Language model processing WikiText
RandomInit (untrained)	0.792	Same architecture, random weights
Random vectors	1.687	Gaussian random $\mathbb{R}^{T \times d}$
Shuffled PT	1.773	PT hidden states with time axis permuted

Table 2: Single-layer (layer 8) NN distance. PT achieves 2.4 $\times$ lower distance than random vectors.

PT produces hidden states from which a linear map yields outputs 2.4 $\times$ closer to real time series than random vectors. Shuffling the temporal dimension destroys this structure (Shuffled $\approx$ Random), confirming that the signal arises from sequential text processing, not static weight properties.

Moving to the concatenated representation ($D = 28,672$) with diversity penalty ($\lambda = 0.5$) dramatically improves both quality and diversity:

	NN Dist	Unique	Clusters
PT concat ($\lambda = 0.5$)	0.667	601 / 10,000	14 / 20
RandomInit concat ($\lambda = 0.5$)	1.723	280 / 10,000	18 / 20

Table 3: Concatenated-layer results with PSD diversity penalty.

5.2 Diversity Requires Both Trained Weights and Meaningful Text

We perform a 2×2 ablation crossing model type (PT vs. RandomInit) with input type (WikiText vs. random tokens). Each condition extracts all 28 layers concatenated ($D = 28,672$), trains a linear mapper with $\lambda = 0.5$ for 100 epochs, and evaluates on both training data ($N = 1920$) and held-out WikiText ($N = 1000$, never seen during training).

Ablation	Loss	Train Unique	Train NN	Held-Out Unique
text_PT	1.33	686	0.672	439
text_RandInit	1.21	54	0.402	73
rand_PT	1.16	4	0.283	1
rand_RandInit	1.29	99	0.522	138

Table 4: 2×2 ablation results. Only text_PT achieves high diversity. The low NN distance of rand_PT is misleading—it reflects mode collapse to 4 unique outputs.

Key findings:

1. **text_PT dominates diversity:** 686 unique matches on training data, 439 on held-out text. No other condition comes close.
2. **Random tokens destroy PT’s diversity:** rand_PT collapses to 4 (train) / 1 (held-out) unique matches. The pretrained model requires meaningful text to produce varied hidden states.

3. **Neither component alone suffices:** Trained weights without meaningful input (rand_PT) collapse. Meaningful input without trained weights (text_RandInit) provides limited diversity. Only the combination succeeds.

Generalization. The mapper trained on WikiText sequences 0–1919 generalizes to held-out sequences 2000–4999:

Split	NN Dist	Unique	Clusters	Top-50
Training ($N = 1920$)	0.667	601	14/20	0.342
Held-out ($N = 3000$)	0.709	760	15/20	0.345

Table 5: Generalization to held-out WikiText. The mapper discovers 760 unique matches on unseen text—more than on training text.

Fair Top- K Comparison. Raw NN distance is misleading because models with fewer unique matches achieve artificially low distances via mode collapse. Table 6 compares models at matched diversity levels.

K	text_PT	text_RandInit	rand_PT	rand_RandInit
4	0.254	0.383	0.330	0.412
54	0.350	0.721	—	0.755
99	0.383	0.825	—	0.862
200	0.441	0.971	—	1.004

Table 6: Top- K fair comparison (mean NN dist of best- K unique matches). “—” indicates fewer than K unique matches available. text_PT wins at every K level.

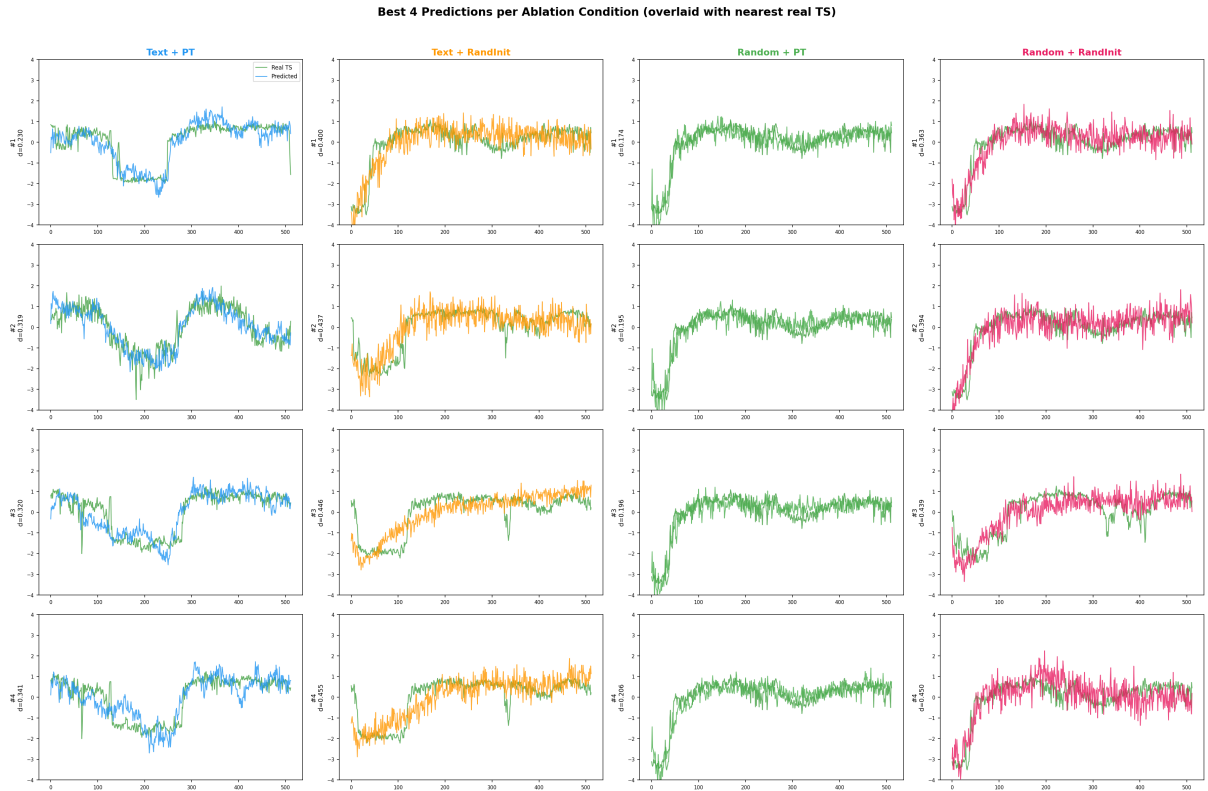
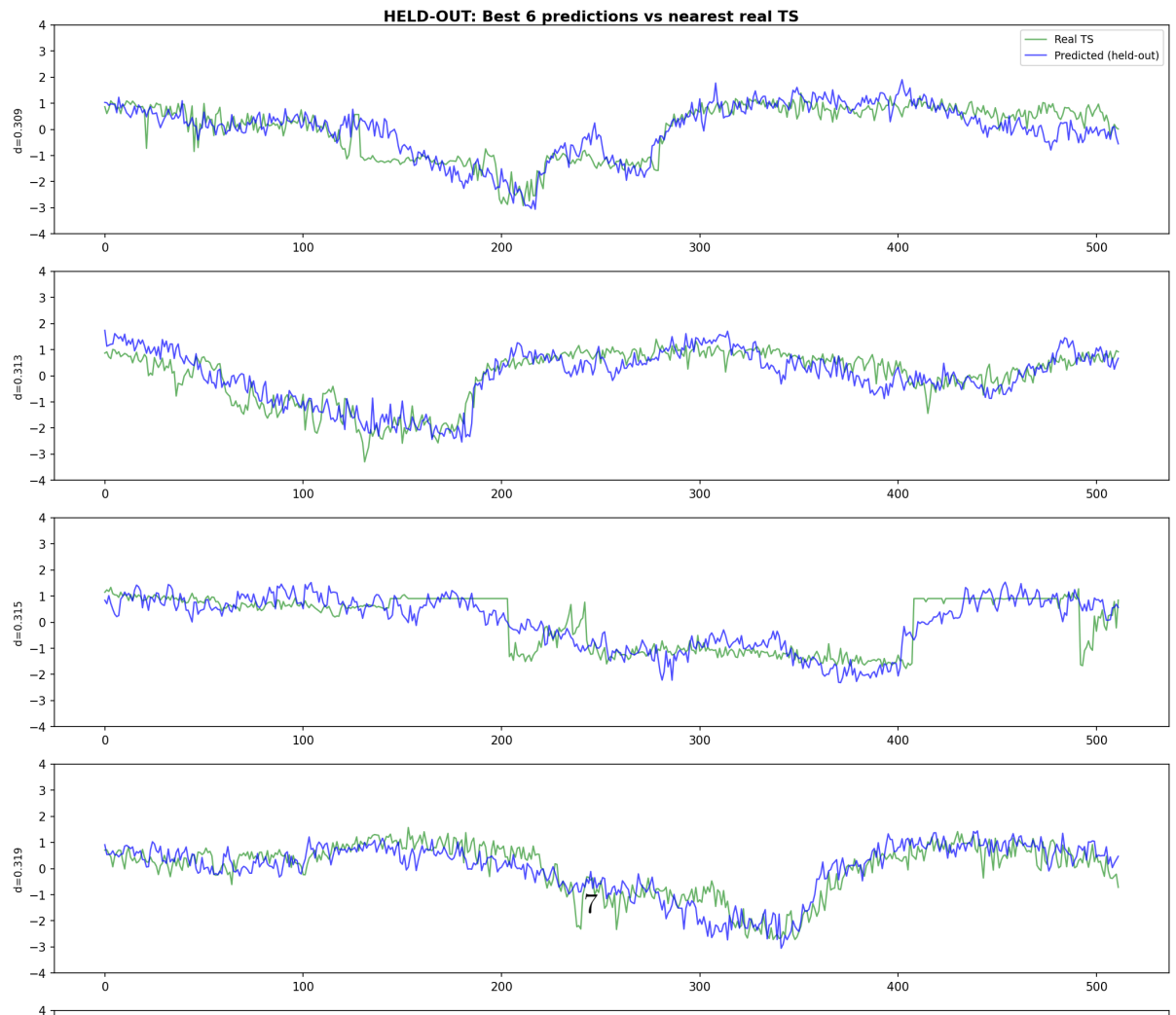


Figure 1: Best 4 predictions per ablation condition overlaid with their nearest real time series (green). text_PT (blue, left) tracks the real TS closely across diverse shapes. rand_PT (green, third column) produces near-identical outputs despite different inputs. RandInit models produce noisy, lower-quality matches.



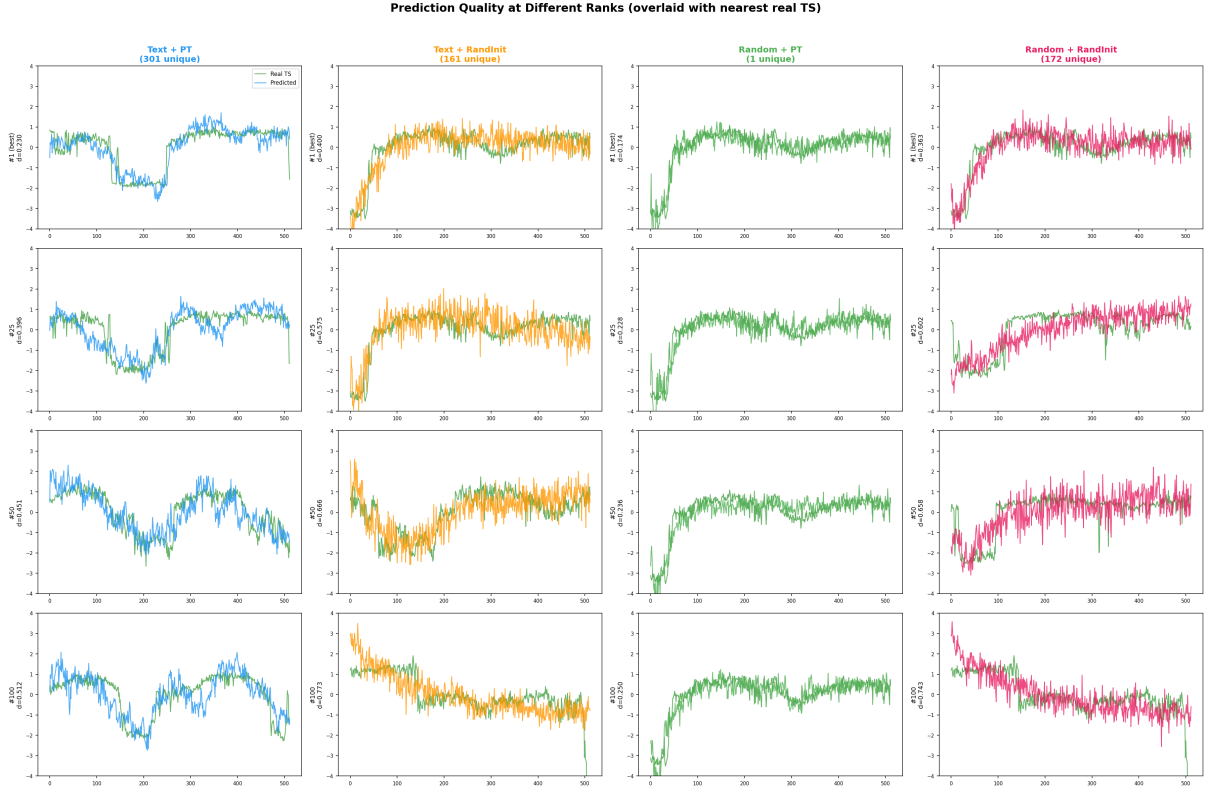


Figure 2: Prediction quality at different ranks ($\#1$, $\#25$, $\#50$, $\#100$ best match) per ablation condition. text_PT maintains good alignment even at rank $\#100$. rand_PT has only 1 unique match (rows 2–4 blank).

6 Representation Geometry

6.1 Random Projection Baseline

To determine whether the alignment between hidden states and time series is intrinsic to representation geometry or purely learned by the mapper, we evaluate *random* linear projections with no training.

For each condition, we sample $R = 50$ random weight vectors $\mathbf{W}_r \sim \mathcal{N}(0, I/D)$, project hidden states:

$$\hat{y}_{i,t}^{(r)} = \mathbf{W}_r \mathbf{h}_{i,t} \quad (13)$$

normalize predictions, and evaluate against the full 10,000 TS bank.

Condition	NN Dist	Unique	Clusters	Entropy
text_PT	1.36 ± 0.30	174 ± 174	12.5 ± 8.2	0.47 ± 0.41
text_RandInit	1.04 ± 0.02	166 ± 10	17.1 ± 0.8	0.63 ± 0.03
rand_PT	1.40 ± 0.34	106 ± 157	9.2 ± 7.8	0.32 ± 0.37
rand_RandInit	1.10 ± 0.02	182 ± 11	17.5 ± 0.7	0.66 ± 0.02

Table 7: Random projection baseline (mean $\pm$ std over 50 projections). PT models show enormous variance across projections; RandInit models are stable.

The most striking result is the *variance structure*: PT models exhibit standard deviations comparable to or exceeding their means (e.g., unique matches 174 ± 174 for text_PT), while RandInit models show tiny relative variance (166 ± 10). This means PT representations contain *rare*,

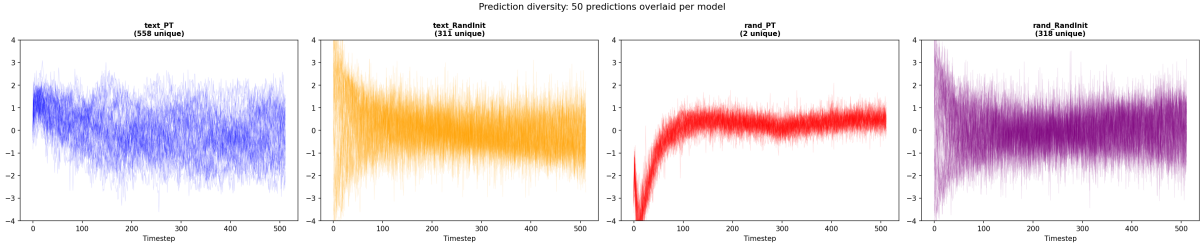


Figure 3: 50 random predictions overlaid per condition. text_PT (top-left) shows rich waveform diversity. rand_PT (bottom-left) collapses to a single shape. RandInit models (right column) show noisy but structurally limited variation.

high-quality directions that random projections occasionally hit, while RandInit representations are isotropic—all directions are equally mediocre.

Comparing to the trained mapper: text_PT’s trained mapper achieves $NN = 0.67$ with 686 unique matches, far surpassing any single random projection. Training finds the optimal direction in a highly structured, anisotropic space.

6.2 PCA Analysis

We directly characterize representation geometry via PCA on flattened hidden states. For each condition, we reshape the hidden state tensor $\mathbf{H} \in \mathbb{R}^{N \times T \times D}$ to $\mathbf{H}_{\text{flat}} \in \mathbb{R}^{NT \times D}$, subsample 50,000 rows, and fit randomized PCA with 500 components.

We report three metrics:

Effective rank. The exponential of the entropy of the normalized eigenvalue spectrum:

$$r_{\text{eff}} = \exp\left(-\sum_{i=1}^D p_i \log p_i\right), \quad p_i = \frac{\lambda_i}{\text{Tr}(\Sigma)} = \frac{\lambda_i}{\sum_{j=1}^D \lambda_j} \quad (14)$$

where λ_i are the eigenvalues of the covariance matrix and $\text{Tr}(\Sigma) = \sum_j \lambda_j$ is the total variance (trace). The effective rank ranges from 1 (all variance in one direction) to D (uniform).

Participation ratio.

$$\text{PR} = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} \quad (15)$$

This measures how many eigenvalues contribute meaningfully to the total variance.

Cumulative variance. The number of PCA components needed to explain a given fraction of total variance.

Condition	Mean Var	Eff. Rank	PR	90%	95%
text_PT	104.4	5.6	1.5	60	315
text_RandInit	3.8	431.3	138.6	421	500+
rand_PT	93.1	2.8	1.3	2	91
rand_RandInit	3.9	585.3	207.6	500+	500+

Table 8: Hidden state geometry. PT models concentrate variance into ~ 3 – 6 effective dimensions; RandInit models spread across 400–600. “90%” / “95%” = number of PCs for that cumulative variance.

PT representations are extremely anisotropic. For text_PT, the first principal component alone captures 81.5% of total variance, and the effective rank is just 5.6 out of $D = 28,672$. For rand_PT, the concentration is even more extreme: 89.2% in the first component, effective rank 2.8.

RandInit representations are near-isotropic. text_RandInit and rand_RandInit have effective ranks of 431 and 585 respectively, with no dominant eigenvalues (the largest explains only 1.7–3.0% of variance).

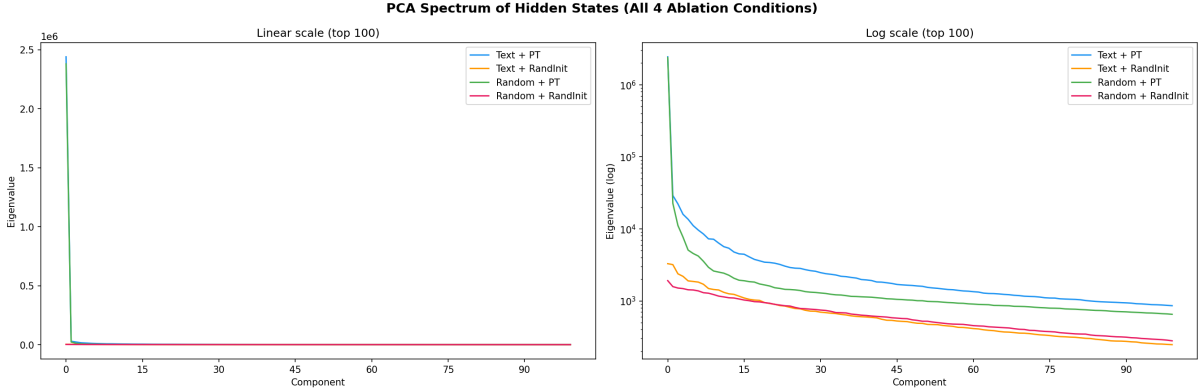


Figure 5: PCA eigenvalue spectrum (top 100 components). Left: linear scale showing the extreme first eigenvalue of PT models. Right: log scale revealing the spectral structure across all conditions.

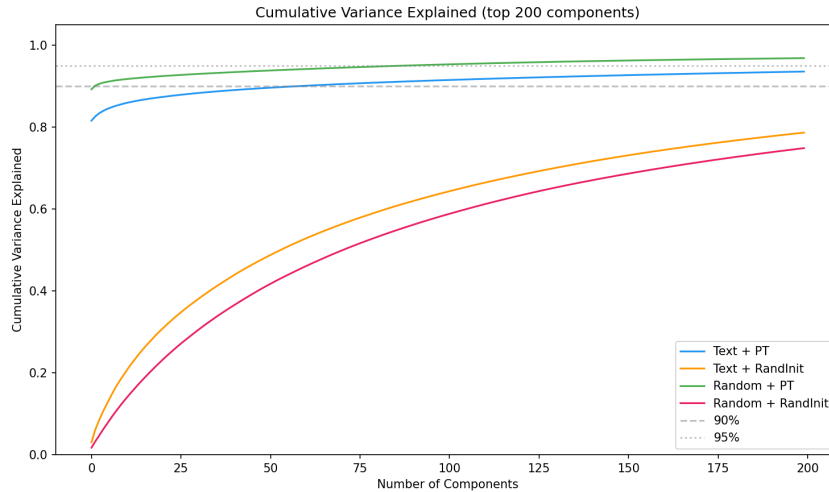


Figure 6: Cumulative variance explained. PT models reach 90% with 2–60 components; RandInit models require 420+ components and still don’t reach 95% within 500.

6.3 Interpretation

The anisotropy of PT representations provides a unified explanation for several observations:

1. **Why linear decoding works:** The pretrained model compresses its representation into a low-dimensional manifold. A linear map can extract useful signal because the relevant information lies in a few accessible directions.
2. **Why random projections are unstable:** With effective rank ~ 5 in a 28,672-dimensional space, a random direction has probability $\sim 5/28,672 \approx 0.02\%$ of aligning with the important subspace. Most random projections miss entirely; rare ones hit.

3. **Why training matters:** The EM-style training procedure finds the optimal direction in this low-dimensional subspace—something random projections cannot reliably do.
4. **Why RandInit is mediocre but stable:** With isotropic geometry (effective rank ~ 500), all directions are equivalent. Training cannot find a better direction because none exists.

7 Spectral Properties

7.1 Power Spectrum Comparison

We compare the frequency content of decoded predictions against real time series. For each sequence $\mathbf{y} \in \mathbb{R}^T$, we compute the normalized PSD:

$$\text{PSD}(\mathbf{y}) = \frac{|\text{FFT}(\mathbf{y})|^2}{\sum_f |\text{FFT}_f(\mathbf{y})|^2} \in \mathbb{R}^{T/2+1} \quad (16)$$

and average across sequences. We measure alignment via L_2 distance and KL divergence between mean PSDs, and decompose energy into frequency bands.

Condition	PSD L_2	KL Div	Low (0–10%)	Mid (10–50%)	High (50–100%)
text_PT	0.228	0.267	90.1%	6.6%	3.3%
text_RandInit	0.274	0.431	58.4%	19.5%	22.1%
rand_PT	0.120	0.125	89.8%	5.8%	4.5%
rand_RandInit	0.264	0.443	55.3%	21.0%	23.7%
Real TS	—	—	79.2%	13.7%	7.2%

Table 9: Spectral alignment. PT models concentrate energy in low frequencies, closely matching real TS. RandInit models produce spectrally flat (noisy) outputs.

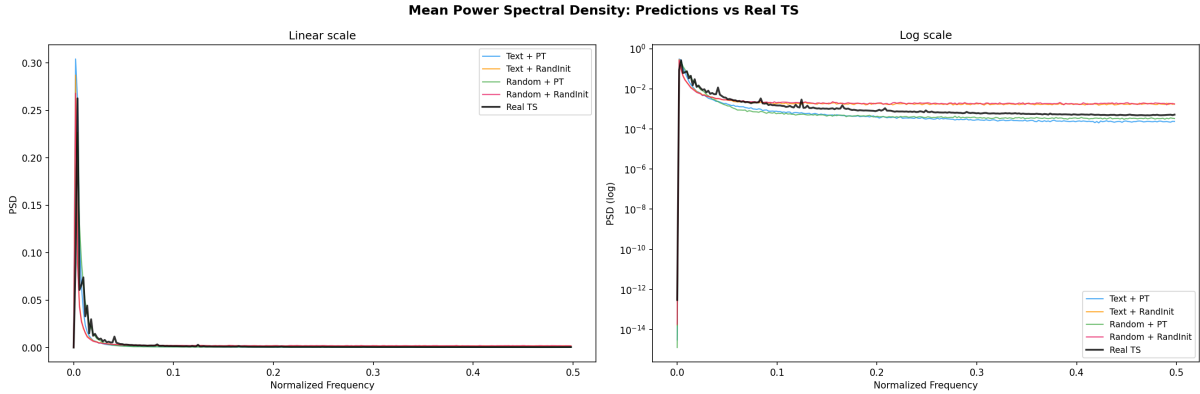


Figure 7: Mean PSD: predictions vs. real TS. Left: linear scale. Right: log scale. PT models (blue, green) match the low-frequency dominance of real TS (black). RandInit models (orange, pink) are spectrally flat.

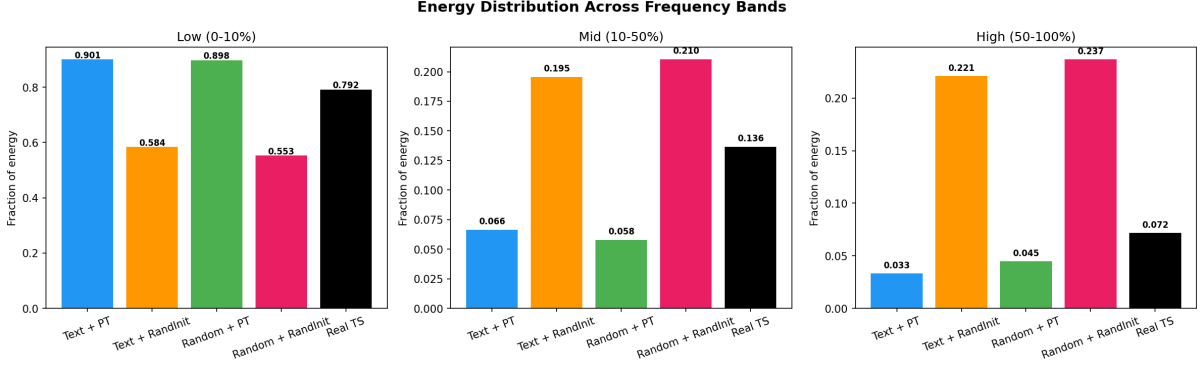


Figure 8: Energy distribution across frequency bands for each condition and real TS.

7.2 Interpretation

PT models produce *smooth, low-frequency-dominated* outputs (90% low-band energy) that closely match the spectral profile of real time series (79% low-band). RandInit models produce spectrally flat, noisy outputs (55–58% low-band, 22–24% high-band).

Notably, rand_PT achieves the *best* spectral alignment (PSD $L_2 = 0.120$) despite collapsing to only 1–4 unique outputs. This demonstrates that spectral realism alone is insufficient—it can be achieved by mode-collapsing to a single representative waveform. Only text_PT achieves both spectral alignment *and* output diversity.

8 Synthesis

8.1 Mechanism

Our results support the following mechanistic model of how linear decoding works:

1. **Pretraining compresses representations** into a low-dimensional subspace (effective rank ~ 3 –6) dominated by 1–2 principal directions.
2. **This subspace encodes smooth temporal dynamics**: the autoregressive training objective induces representations that vary smoothly along the sequence dimension, producing low-frequency-dominated outputs when linearly projected.
3. **Text input traverses the manifold**: different text sequences move the hidden state through different trajectories on this low-dimensional manifold, enabling diverse outputs.
4. **Linear decoding projects these trajectories** into one-dimensional time series that resemble real-world temporal patterns.

8.2 Why Each Component Is Necessary

- **Trained weights**: Create the low-dimensional manifold with smooth temporal dynamics. Without training, representations are isotropic and all projections yield noise.
- **Meaningful text**: Drives the hidden state through varied trajectories on the manifold. Random tokens produce uniform hidden states that collapse to a single output.
- **The linear map**: Extracts the right direction in the low-dimensional subspace. Random projections occasionally find good directions but cannot do so reliably in a 28,672-dimensional space with effective rank ~ 5 .

8.3 Failure Modes

- **Mode collapse**: The primary failure mode. The EM objective naturally rewards matching the most common TS pattern. Mitigated by the PSD diversity penalty ($\lambda = 0.5$), but not

eliminated.

- **Seed sensitivity:** Results vary across random seeds (601–883 unique matches across seeds 0–4). Some seeds collapse entirely, likely due to early EM assignments trapping the mapper in a local minimum.
- **Over-smoothing:** PT predictions concentrate 90% of energy in low frequencies (vs. 79% for real TS), under-representing mid/high frequency variation.

9 Conclusion

We have shown that pretrained language models encode latent temporal structure that can be linearly decoded into realistic time series without any paired supervision. This structure is:

- **Low-dimensional:** effective rank ~ 3 –6 in a 28,672-dimensional space, with 80–89% of variance in a single direction.
- **Spectrally aligned:** decoded outputs match the low-frequency dominance of real time series (90% vs. 79% low-band energy).
- **Dependent on both training and semantics:** neither trained weights alone nor meaningful text alone produces diverse, high-quality outputs.

These findings suggest that autoregressive language pretraining induces a latent dynamical system in representation space—one whose temporal structure is rich enough to be linearly projected into observable time series signals. The extreme anisotropy of pretrained representations (effective rank ~ 5 despite 28,672 dimensions) implies that language models learn to compress sequential information into a remarkably compact subspace, and it is this compression that enables—and constrains—cross-modal transfer.

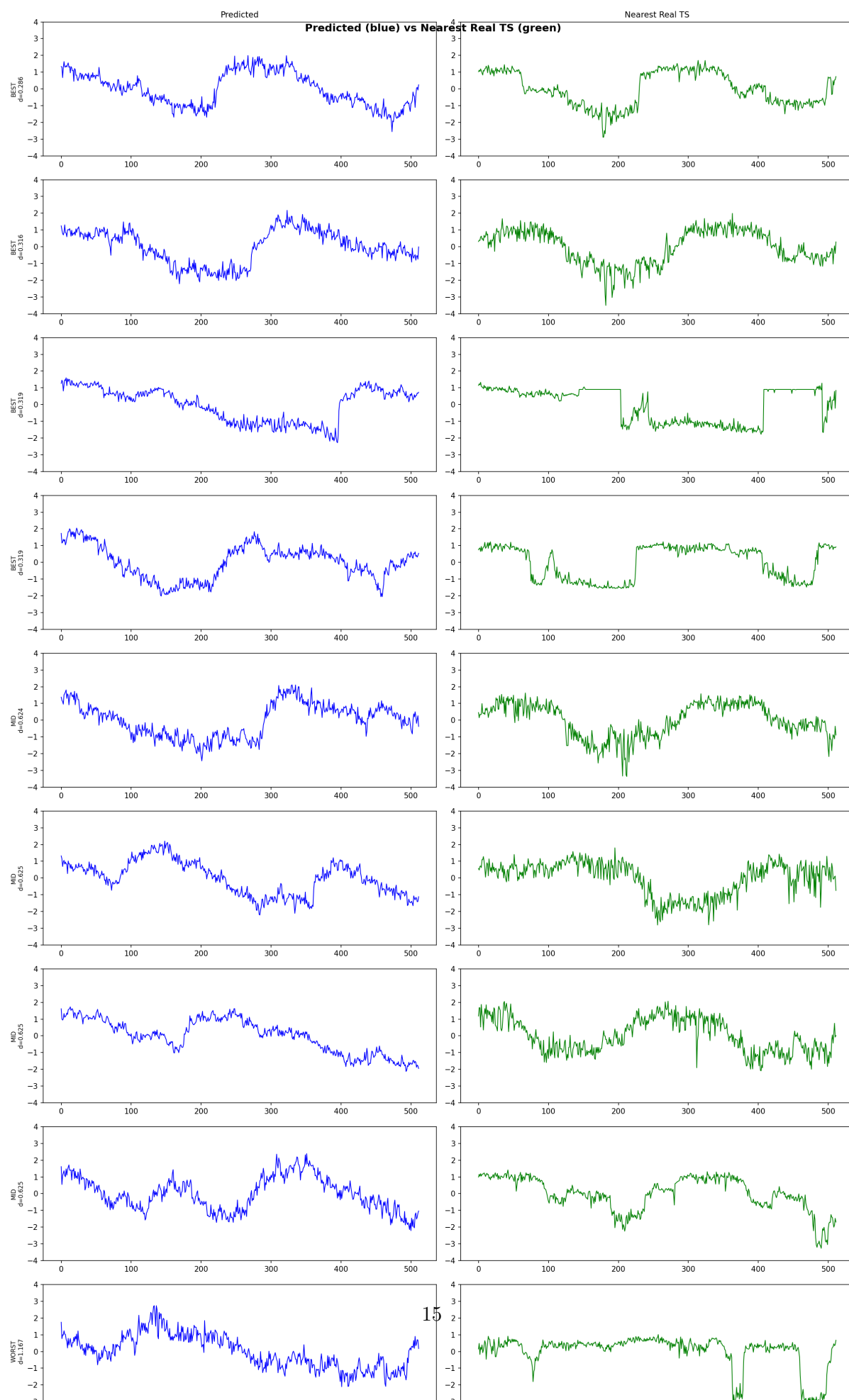
A Additional Training Details

Infrastructure. All experiments were conducted on a machine with $4\times$ NVIDIA RTX 5090 (32 GB each) and 755 GB RAM. Hidden state extraction (28 layers, 2000 sequences) takes ~ 3 minutes. Mapper training (100 epochs, batch 32, concat layers) takes ~ 45 minutes. NN evaluation (N predictions $\times$ 10,000 TS bank) takes ~ 8 minutes.

Hidden state storage. Concatenated hidden states for 2000 sequences ($2000 \times 512 \times 28,672$ in float16) require ~ 55 GB. We store these as float16 numpy memmaps on disk, loading batches to float32 on GPU during training.

Seed stability. We tested 5 random seeds (0–4) with the text_PT concatenated configuration. Unique matches ranged from 601 to 883 across seeds, with seed 2 exhibiting partial collapse. Seed 1 was selected for all subsequent experiments based on stability.

B Additional Results



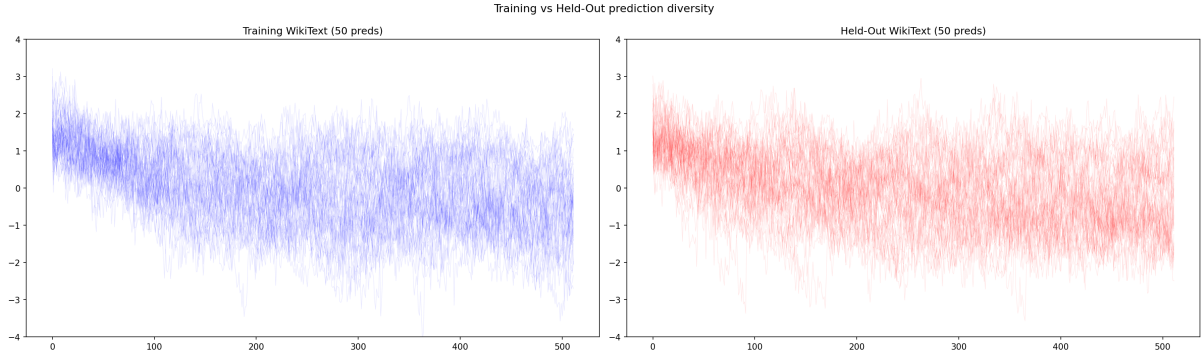


Figure 10: Training (left) vs. held-out (right) prediction diversity for text_{PT}. Both show similar waveform variety, confirming generalization.

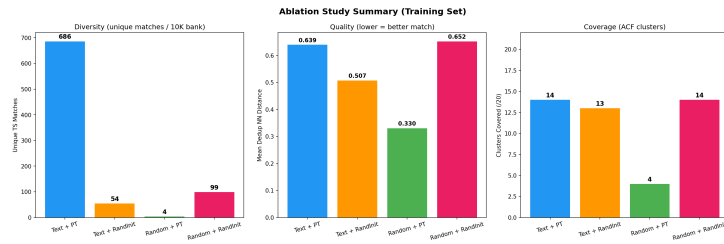


Figure 11: Summary of ablation study: diversity (unique matches), quality (dedup NN distance), and cluster coverage across all four conditions.